

Network Timer Challenge

Building Your First Network Variable in Unity



MCAST

Institute of Information &
Communication Technology

 david.deguara@mcast.edu.mt



Learning Objectives

What You'll Learn Today

- Understanding Network Variables in Netcode for Game Objects
- Synchronising data between server and clients
- Implementing server authority for game logic
- Creating a countdown timer for multiplayer games

Why Network Variables?

- Automatic synchronisation across all clients
- Simple to implement and understand
- Server-authoritative by default
- Perfect for shared game state

The background is a complex, abstract composition. It features large, overlapping organic shapes in shades of red, orange, purple, and blue. Interspersed among these are patterns of fine, concentric lines in pink and blue, and rectangular areas filled with a grid of small white dots on a dark background. A semi-transparent horizontal band is positioned across the middle of the image.

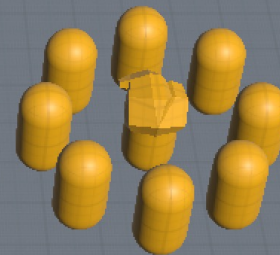
The Challenge

Your Task (30 Minutes)

Create a **Network Timer** that:

1. Starts when **2 players** join the match
2. Counts down from **8 seconds** to **0**
3. Stops at 0 (doesn't go negative)
4. Displays synchronised time on all clients

8.00



Q - 140% +

position (6.00, 1.00, -2.00)

[13:07:51] Spawned unit 8/8 for client 0 at

position (7.41, 1.00, -1.41)

[13:07:51] New player joined! Client ID: 0. Total
connected players: 1

[13:07:51] Player for client 0 already spawned!

[13:07:51] Host started on port 7777

Requirements

- Use a `NetworkVariable<float>` to store the timer
- Only the **server** should update the timer
- All clients should see the **same value**
- Display the timer using **TextMeshPro UI**





Network Variables Basics

Declaring a Network Variable

```
private NetworkVariable<float> myValue = new  
NetworkVariable<float>(  
    defaultValue: 8f,  
    NetworkVariableReadPermission.Everyone,  
    NetworkVariableWritePermission.Server  
) ;
```

Key points:

- Everyone can read
- Only server can write
- Automatically syncs to all clients

Accessing the Value

```
// Reading the value
float currentValue = myValue.Value;

// Writing the value (server only!)
if (IsServer)
{
    myValue.Value = 5f;
}
```



Server Authority Pattern

Checking If You're the Server

```
private void Update()  
{  
    if (!IsServer) return;  
  
    // Only server code runs here  
    // Update game logic, timers, etc.  
}
```

Why? Only one machine should control the timer to avoid conflicts!

Checking Connected Players

```
int playerCount =  
NetworkManager.Singleton.ConnectedClients.Count  
;  
  
if (playerCount >= 2)  
{  
    // Two players have joined!  
}
```

The background is a vibrant, abstract composition. It features large, overlapping organic shapes in shades of red, orange, purple, and blue. Interspersed among these are patterns of fine, concentric lines and grids of small white dots on darker backgrounds. The overall effect is dynamic and modern.

Implementation Hints

Structure Your Script

You'll need:

1. A `NetworkVariable<float>` for the timer (initialised to 8)
2. A `bool` to track if the timer has started
3. A `SerializedField` for the `TextMeshProUGUI` label
4. Logic in `Update ()` to countdown (server only)
5. Logic in `LateUpdate ()` to update the UI

Countdown Logic Hint

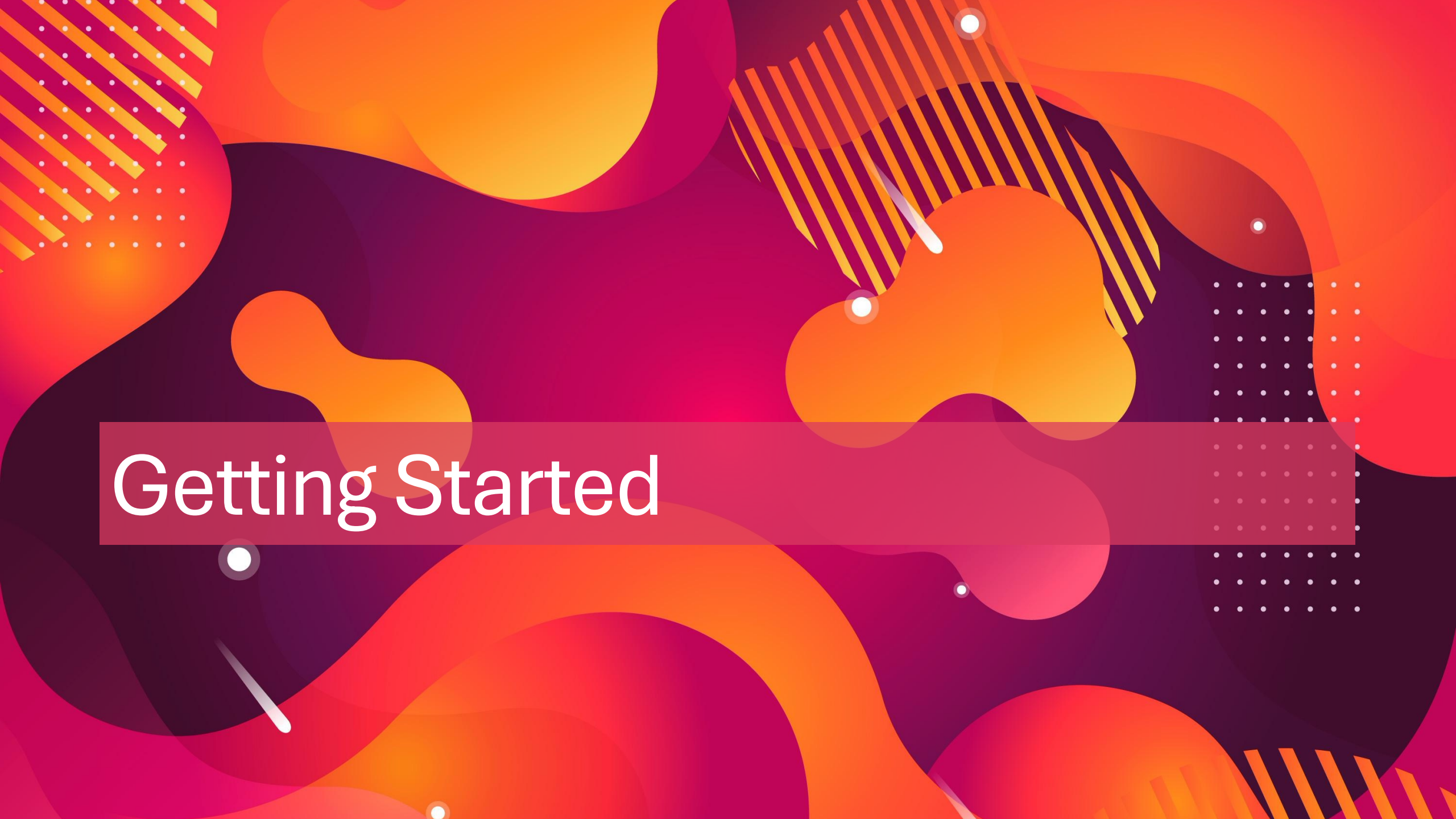
```
if (timerValue.Value > 0)
{
    timerValue.Value -= Time.deltaTime;

    if (timerValue.Value <= 0)
    {
        timerValue.Value = 0;
        // Timer finished!
    }
}
```

UI Update Hint

```
private void LateUpdate()  
{  
    if (timerLabel != null)  
    {  
        timerLabel.text = $"{  
            timerValue.Value:F2}s";  
    }  
}
```

The :F2 formats the float to 2 decimal places.



Getting Started

Setup Steps

1. Create a new C# script called `NetworkTimer`
2. Inherit from `NetworkBehaviour` (**not** `MonoBehaviour`!)
3. Add required using statements:
 - `using Unity.Netcode;`
 - `using TMPro;`
4. Attach script to a `GameObject` with a `NetworkObject` component
5. Create a Canvas with TextMeshPro UI element
6. Assign the UI element to your script

Testing Your Timer

- Build and run as Host
- Start a second instance as Client
- Both should see the same countdown
- Timer should start when 2nd player joins
- Timer should stop at exactly 0



Success Criteria

Your Timer Should:

- ✓ Start automatically when 2 players connect
- ✓ Count down from 8.00s to 0.00s
- ✓ Show the same value on both clients
- ✓ Stop at 0 (not go negative)
- ✓ Display in a TextMeshPro UI element